PureRoute v2 Decision Engine Playbook

Nov 2, 2026 · @siddhi

The system has two parts:

- **Arduino Uno** reads temperature, flow, turbidity and conductivity once a second, runs the decision rules and the fail-safe, and drives the pump and the four LEDs (LEDs stand in for the purifier's valves). It keeps working safely even if the laptop disconnects.
- **Laptop** (Windows, Mac or Linux; a Raspberry Pi also works), connected to the Uno by USB, runs the Python dashboard, CSV logging, and the area lock and test-result unlock, and sends the lock on or off to the Uno.

The output is one action: route to UF+UV or RO+UV, flush, replace the filter, force RO because of an area lock, or stop safely on a fault.

Inputs

The Uno prints one CSV line per second over USB (115200 baud). Everything the engine decides and logs comes from these columns:

```
t_ms,temp_c,flow_lpm,pulses,turb_mv,turb_index_pct,ec25_uScm,tds_ppm,fhi,counter_life_pct,pump,lock,state,reason,tag
```

Value	How it is calculated	Notes
`temp_c`	DS18B20, 11-bit, non-blocking read	85 °C = read too early; −127 °C = wiring fault
`flow_lpm`	Pulses counted by interrupt each second: Q = pulses × 60 / K	K comes from the bucket calibration (expect 4,000–6,000 pulses/L); listings disagree, so never hard-code 5,880
`turb_index_pct`	TI = 100 × (1 − V / V0), V0 = clear-water voltage at session start	Relative index only. Never report NTU.
`ec25_uScm`	Bipolar pulses on the homemade probe give resistance Rx; G = 1/Rx; EC = a + b·G + c·G² from the salt calibration; then EC25 = EC / (1 + 0.019 × (T − 25))	Realistic accuracy ±5–10% from about 100 to 1,500 µS/cm after calibration

Value	How it is calculated	Notes
`tds_ppm`	EC25 × 0.6	State the factor (0.5–0.7 are all used)
`fhi`	Filter Health Index, see below	Our core metric
`counter_life_pct`	Litre counter, the way purifiers track filter life today	For contrast in the demo

Signal cleaning:

- Read each analog channel 15 times and use the median.
- The Uno's ADC is 10-bit on a 5 V reference: mV = reading × 5000 / 1023. Measure the real 5 V rail with a multimeter and use that value. Only trust readings between about 100 and 4,900 mV.
- All Uno inputs accept 5 V, so the turbidity and flow signals need no voltage dividers.
- Smooth flow and FHI with an EMA (α = 0.2).
- Store the calibration constants (K, V0, Q0, EC coefficients) in the Uno's EEPROM, so a reboot doesn't lose them. Each unit keeps its own constants.
- The Uno has only 2 KB of RAM: keep reason strings short and wrap fixed text in `F("...")`.

Decision rules

The engine checks the states top to bottom and takes the first that applies. A state is entered only after its condition holds for 5 seconds in a row, and left only when the exit condition holds. That stops the LEDs flickering. Thresholds are demo values to tune from your own samples.

Priority	State	LED	Pump (stands in for dispensing)	Enter when	Exit when	Example reason string
1	FAULT	Red	**OFF (dispense held)**	Any fault flag (next section)	Flag clear for 5 s	`NO FLOW 4s with pump ON -> dry run; pump OFF`
2	LOCKED-RO	Blue, blinking	ON	Area flag set and no valid test on file	Valid test entered	`District flag: As; no test on file -> RO forced`
3	REPLACE	Yellow, solid	ON	FHI < 40, or flow < 0.3 L/min with pump ON	FHI > 50	`FHI=35<40 (Q=0.42/Q0=1.10) -> REPLACE filter`

Priority	State	LED	Pump (stands in for dispensing)	Enter when	Exit when	Example reason string
4	FLUSH	Yellow, blinking	ON, route RO	TI > T_hi (turbidity spike)	TI < T_hi − margin	`TI=38%>25% -> FLUSH prefilter, protect UF/RO`
5	RO	Blue	ON	EC25 > 500 µS/cm (about 300 ppm, Intellon's switch point), or EC reading uncertain	EC25 < 450	`EC25=812>500 -> RO+UV`
6	UF+UV	Green	ON	EC25 ⩽ 500 and TI < T_lo and no lock	—	`EC25=210, TI=3% -> UF+UV (RO skipped)`

Starting thresholds (set from your own clean/muddy runs on 3 Nov):

- **T_lo:** 10% turbidity index
- **T_hi:** 25%
- **Margin:** 5 points
- **EC switch:** 500 µS/cm in, 450 out

Two rules always win: a fault stops dispensing, and any doubt about the water picks RO.

Fail-safe and fault detection

On any fault: the pump stops (no dispensing), the red LED lights, the dashboard shows the reason, and the route resets to RO when the fault clears.

Fault	Check	Bench test to prove it
Dry run / no flow	Pump ON and flow < 0.05 L/min for more than 3 s	Lift the pump out of the water
Leak or siphon	Flow > 0.1 L/min with pump OFF for more than 3 s	Lift the outlet tube above the reservoir to siphon
Temperature sensor	Reads −127 or 85 °C, or outside 0–50 °C	Unplug the DS18B20 data wire
Turbidity sensor	ADC below 100 mV or above 2,400 mV	Unplug its signal wire

Fault	Check	Bench test to prove it
Conductivity probe in air	Rx above 200 kΩ	Lift the probe out of the cup
Conductivity probe shorted	Rx below 50 Ω	Touch the two screws together
Probe fouled or polarised	The two pulse directions disagree by more than 20%	Leave probe in salty water without cleaning (slow)
Stuck sensor	A reading doesn't change at all over 30 samples while the pump runs	Hold a fixed voltage on a pin (optional)
Flow sensor dead	Looks the same as a dry run (pump ON, no pulses), so it also stops the pump	Unplug the flow signal wire

For each fault, log the detection time (seconds from fault to red LED) and the recovery time.

Filter Health Index

Filter health is calculated from what the filter is actually doing (flow through it), not from how many litres have passed. A litre counter runs alongside for comparison.

```
FHI = 100 × clamp(Q / Q0, 0, 1)        smoothed with EMA, α = 0.2
Q0  = flow with a fresh cartridge, stored with the F command
counter_life = 100 × (1 − litres / rated_litres)   rated_litres = 50 L on the bench
```

FHI	Meaning	State
70–100	Healthy	Normal routing
40–70	Clogging	Dashboard warns "flush soon"
Below 40	Clogged	REPLACE (yellow)

Expected behaviour: as the cartridge clogs, FHI falls from 100 to below 40 while the litre counter stays near 95%. Plot both on the same chart.

Optional extra (only if time): fit a straight line to FHI over the last few minutes and show "about N minutes to REPLACE" on the dashboard. That is a real, simple prediction.

Area lock and evidence-based unlock

A flagged district forces RO. **A dated test result can unlock it, with an expiry.**

How it works:

1. A small `district_risk.csv` (district, As, F, NO3, U flags) is entered by hand from the CGWB Annual Ground Water Quality Report 2024 tables: fluoride Table 11, nitrate Table 18, arsenic Table 21, uranium Table 24. Five to ten districts are enough for the demo.
2. The user picks a district and a source (groundwater or piped). The lock applies only to groundwater sources in a flagged district.
3. A district not in the table shows **"no flag found"**, never "safe". Nainital has no CGWB entry, so this is the expected result for Haldwani.
4. A test result is entered in the dashboard form. It must carry the source (lab report, Jal Jeevan Mission field kit, or village result), a date and values.
5. It unlocks only if **every flagged parameter** is within the BIS limits CGWB uses:
   - arsenic below 10 µg/L
   - fluoride 1.5 mg/L or less
   - nitrate 45 mg/L or less
   - uranium 30 µg/L or less
6. The unlock expires after 6 months. For nitrate flags, it also re-locks after the monsoon, because CGWB saw nitrate exceedance rise after the monsoon.
7. Test strips are **advisory only** and never unlock arsenic. One field study found 54% false negatives for arsenic strips.

Example reason strings:

- `District flag: As; no test on file -> RO forced`
- `Test 2026-11-04 (lab) As=18ug/L > 10 -> RO stays locked`
- `Unlocked: lab 2026-11-04 As=4ug/L; expires 2027-05-03`

Firmware skeleton

One `millis()` loop at 1 Hz on the Uno, with the flow interrupt counting all the time. This is a starting scaffold, not tested code. Pins match the Hardware Playbook.

```
#include <OneWire.h>
#include <DallasTemperature.h>
#include <EEPROM.h>
#include <avr/wdt.h>

const byte PIN_T=4, PIN_FLOW=2, PIN_TURB=A0, PIN_EC=A1, EC_A=7, EC_B=8,
```

```
EC_LO=9;
const byte PIN_PUMP=5, LED_UF=10, LED_RO=11, LED_YEL=12, LED_RED=13;
const float R_SER=1000.0, VCC_MV=5000.0;      // measure your 5V rail and
edit
const float T_HI=25, T_LO=10;                 // turbidity index thresholds
(%)
struct Cal { float kFlow, v0, q0, ecA, ecB, ecC; } cal;   // saved in EEPROM

OneWire ow(PIN_T); DallasTemperature ds(&ow);
volatile unsigned long flowCount=0;
void onPulse(){ flowCount++; }
bool locked=false;                            // set by the dashboard with L1
/ L0

float medianMv(byte pin){ int v[15];
  for(byte i=0;i<15;i++) v[i]=analogRead(pin);
  for(byte i=1;i<15;i++){ int k=v[i]; int j=i-1; while(j>=0&&v[j]>k)
{v[j+1]=v[j];j--;} v[j+1]=k; }
  return v[7]*VCC_MV/1023.0; }

bool ecBurst(float &rx, float &asym){         // equal +/- pulses, no net DC
  pinMode(EC_A,OUTPUT); pinMode(EC_B,OUTPUT);
  float s1=0,s2=0; byte n1=0,n2=0;
  for(byte i=0;i<8;i++){
    digitalWrite(EC_A,HIGH); digitalWrite(EC_B,LOW);  delayMicroseconds(400);
    float v1=analogRead(PIN_EC)*VCC_MV/1023.0;
    digitalWrite(EC_A,LOW);  digitalWrite(EC_B,HIGH); delayMicroseconds(400);
    float v2=analogRead(PIN_EC)*VCC_MV/1023.0;
    if(v1>100&&v1<4900){ s1+=R_SER*v1/(VCC_MV-v1); n1++; }
    if(v2>100&&v2<4900){ s2+=R_SER*(VCC_MV-v2)/v2; n2++; }
  }
  digitalWrite(EC_A,LOW); digitalWrite(EC_B,LOW); pinMode(EC_A,INPUT);
pinMode(EC_B,INPUT);
  if(!n1&&!n2) return false;
  float r1=n1?s1/n1:NAN, r2=n2?s2/n2:NAN;
  rx=(n1&&n2)?(r1+r2)/2:(n1?r1:r2);
  asym=(n1&&n2)?fabs(r1-r2)/rx:0; return true;
}

enum State {UF, RO, FLUSH, REPLACE, LOCKED_RO, FAULT};

State decide(bool fault, float fhi, float q, bool pumpOn, float ti, float
ec25, bool ecOk){
  if(fault)                       return FAULT;      // pump OFF
  if(locked)                      return LOCKED_RO;
  if(fhi<40 || (pumpOn && q<0.3))  return REPLACE;
```

```
  if(ti>T_HI)                       return FLUSH;
  if(!ecOk || ec25>500)             return RO;
  return UF;
}

void setup(){
  pinMode(PIN_PUMP,OUTPUT); digitalWrite(PIN_PUMP,LOW);   // pump off first
  Serial.begin(115200); ds.begin(); ds.setWaitForConversion(false);
  EEPROM.get(0,cal);
  pinMode(PIN_FLOW,INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(PIN_FLOW), onPulse, FALLING);
  wdt_enable(WDTO_2S);                      // reset if the loop hangs

Serial.println(F("t_ms,temp_c,flow_lpm,pulses,turb_mv,turb_index_pct,ec25_uScm,tds_ppm,fhi,counter_life_pct,pump,lock,state,reason,tag"));
}
// loop(): wdt_reset(); every 1000 ms read sensors, run faults(), decide(),
// 5-second persistence + hysteresis, setLEDs(state), pump OFF in FAULT,
// print one CSV line, parse serial commands, save calibration to EEPROM.
```

Serial commands (from the dashboard or the Serial Monitor):

Command	Action
`B`	Save clear-water turbidity baseline V0
`F`	Save fresh-cartridge flow Q0
`K<n>`	Set flow constant (pulses per litre) from the jug calibration
`P1` / `P0`	Pump on / off
`S<name>`	Tag the current sample (written in every CSV row)
`L1` / `L0`	Area lock on / off (sent by the dashboard's lock module)

Dashboard

The dashboard reads the ESP32's serial output, logs every row to CSV and shows live charts plus the current state and its reason.

Step 1, about 1–2 hours: matplotlib (`pip install pyserial matplotlib`).

- A reader thread saves every valid line to `run_YYYYMMDD_HHMM.csv` and keeps the last 600 rows.

- Four panels: EC25; turbidity index; FHI with the dashed litre counter on the same axis; temperature.
- The title shows `STATE: reason` in large text.
- Skip lines that don't have all 15 columns, such as the boot banner.
- Close the Arduino Serial Monitor first, because only one program can hold the COM port.

Step 2, 3–4 hours, only if Step 1 works: Streamlit.

- Metric tiles and line charts.
- A big coloured state badge.
- A reason log table.
- Sidebar buttons that send `B / F / P1 / P0 / S`.
- A lock panel with a district picker, a source picker and a "submit test result" form.
- Keep the serial port in `st.cache_resource`, or reruns will report the port as busy.

Starting script for Step 1 (set `PORT` to your COM port):

```
# dashboard_min.py  (pip install pyserial matplotlib)
import serial, threading, collections, time, csv
import matplotlib.pyplot as plt, matplotlib.animation as anim
PORT, BAUD = "COM3", 115200   # Windows: COMx from Device Manager; Mac/Linux:
/dev/ttyACM0
HDR =
"t_ms,temp_c,flow_lpm,pulses,turb_mv,turb_index_pct,ec25_uScm,tds_ppm,fhi,cou
nter_life_pct,pump,lock,state,reason,tag".split(",")
buf = collections.deque(maxlen=600)
ser = serial.Serial(PORT, BAUD, timeout=1)
def reader():
    with open(time.strftime("run_%Y%m%d_%H%M.csv"), "w", newline="") as f:
        w = csv.writer(f); w.writerow(HDR)
        while True:
            parts = ser.readline().decode(errors="ignore").strip().split(",")
            if len(parts) != len(HDR) or parts[0] == "t_ms": continue
            w.writerow(parts); f.flush(); buf.append(dict(zip(HDR, parts)))
threading.Thread(target=reader, daemon=True).start()
fig, ax = plt.subplots(4, 1, figsize=(9, 9), sharex=True)
def draw(_):
    if not buf: return
    rows = list(buf); x = [int(r["t_ms"]) / 1000 for r in rows]
    for a in ax: a.clear()
    ax[0].plot(x, [float(r["ec25_uScm"]) for r in rows]);
ax[0].set_ylabel("EC25 uS/cm")
    ax[1].plot(x, [float(r["turb_index_pct"]) for r in rows]);
```

```
ax[1].set_ylabel("Turb index %")
    ax[2].plot(x, [float(r["fhi"]) for r in rows], label="FHI (measured)")
    ax[2].plot(x, [float(r["counter_life_pct"]) for r in rows], "--",
label="litre counter")
    ax[2].legend(); ax[2].set_ylabel("Filter health %")
    ax[3].plot(x, [float(r["temp_c"]) for r in rows]); ax[3].set_ylabel("T
degC")
    fig.suptitle(f'{rows[-1]["state"]}: {rows[-1]["reason"]}', fontsize=16)
a = anim.FuncAnimation(fig, draw, interval=1000); plt.show()
```

Logic tests and what to log

Run these once the rig works. Record mean ± SD and the number of runs, never a single best reading.

Test	How	Runs	Record
Clogging ramp	Fresh → half → clogged cartridge, 3 min each	3	FHI and litre-counter curves; seconds to REPLACE
Classification	Clean, salty 1 g/L, salty 2 g/L, muddy, salty + muddy, 30 s each	5 each	Table of expected vs actual LED; detection rate = correct / total; false alarms on clean water
Lock logic	Set As flag → failing test → passing test → move the clock past expiry	3	Correct state at every step, with reason strings
Fault injection	Every row of the fail-safe table	5 each	Detected yes/no, detection time, recovery time
Repeatability	Same tap sample, probes removed and replaced	10	EC25 and TI mean ± SD, CV%

Save every run's CSV with its sample tag. The hardware team runs the calibrations; the software team owns these logic tests.

Build order

- [ ] Install the Arduino IDE on the laptop with the OneWire and DallasTemperature libraries
- [ ] Write the Uno firmware from the skeleton, with a fake-sensor mode to test the logic before the rig is ready

- [ ] On the laptop: `pip install pyserial matplotlib`, set `PORT` to the Uno's port and get `dashboard_min.py` plotting from the fake mode
- [ ] Create `district_risk.csv` with 5–10 districts from the CGWB 2024 report, including at least one arsenic and one nitrate district
- [ ] Write the dashboard's lock module: district + source + test result → sends `L1` or `L0` to the Uno
- [ ] Flash onto the real rig after the hardware bring-up
- [ ] Load the calibration constants (K, V0, Q0, EC coefficients) into EEPROM
- [ ] Make every state reachable: UF, RO, FLUSH, REPLACE, LOCKED-RO, FAULT
- [ ] Tune T_lo, T_hi and the EC threshold from clean, salty and muddy runs
- [ ] Add the fault checks and confirm the pump stops on each, including when the laptop is unplugged
- [ ] Run the logic tests and save every CSV
- [ ] Optional: Streamlit dashboard and the FHI time-to-replace estimate
- [ ] Freeze the firmware and tag it `v2`